

Colab Orchestrator Platform

i want to create a application ,that can do this
we can inserte jupeyter notebook files in to this and this application connect with the google colabe and train the model and save the model to the application using this application we can download the model

You're basically describing a **"Colab Orchestrator Platform"** — a system that:

1. Accepts **Jupyter Notebooks (.ipynb)**
2. Sends them to **Google Colab for execution**
3. Runs training (GPU/TPU)
4. Extracts the trained model
5. Stores it inside your system
6. Lets users **download / manage models**

This is a **very powerful SaaS idea** (and honestly aligns with your AI/agent systems thinking 🔥)

High-Level System Idea

Core Concept

"Upload notebook → Run on Colab → Get trained model back automatically"

FULL SYSTEM ARCHITECTURE

Main Components

```
Frontend (React)
  ↓
Backend API (FastAPI / Node)
  ↓
Colab Orchestrator Engine
  ↓
Google Colab Runtime (via automation)
  ↓
Model Storage (S3 / Local / GDrive)
```

HOW IT ACTUALLY WORKS (CORE FLOW)

Step-by-step execution

1. Upload Notebook

- User uploads **.ipynb**

- Backend parses:
 - Detect training code
 - Detect output file paths

2. Inject Runtime Code (IMPORTANT 🔥)

You modify notebook automatically:

```
# Inject at top
from google.colab import drive
drive.mount('/content/drive')

# Inject at end
import shutil
shutil.copy("model.pth", "/content/drive/MyDrive/models/model.pth")
```

3. Send to Colab

You have 3 possible approaches

OPTION A — Browser Automation (BEST START)

Use:

- Playwright / Puppeteer

Flow:

1. Open colab notebook
2. Upload `.ipynb`
3. Click "Run All"
4. Wait until finished
5. Download model

OPTION B — Colab API (LIMITED)

Google doesn't officially allow:

- Remote execution control easily

So this is harder.

OPTION C — Jupyter Kernel Bridge (ADVANCED)

- Use Colab as backend kernel
- Connect via:

- WebSockets
- Notebook execution APIs

👉 Hard but scalable

ORCHESTRATOR ENGINE (YOUR SECRET WEAPON)

This is your core innovation.

Responsibilities:

◆ Notebook Analyzer

- Detect:
 - Model save path
 - Dataset loading
 - GPU usage

◆ Code Injector

- Auto-add:
 - Google Drive sync
 - Logging
 - Progress tracking

◆ Execution Tracker

- Monitor:
 - Training progress
 - Errors
 - GPU usage

◆ Result Extractor

- Pull:
 - .pt, .h5, .ckpt
 - Metrics
 - Logs

DATABASE DESIGN

Tables

Users

```
id
email
plan
```

Notebooks

```
id
user_id
file_path
status (pending, running, done)
```

Jobs

```
id
notebook_id
start_time
end_time
status
logs
```

Models

```
id
job_id
model_path
framework (pytorch/tensorflow)
size
```

WORKER SYSTEM (VERY IMPORTANT)

Use:

-  RabbitMQ (you already explored this)
- or Redis Queue

Flow:

Upload → Queue → Worker → Run on Colab → Save result

FRONTEND (React)

Screens

1. Dashboard

- Jobs
- Status
- GPU usage

2. Notebook Upload

- Drag & drop .ipynb

3. Job Viewer

- Logs (live streaming 🔥)
- Progress bar

4. Model Manager

- Download models
- Versioning

AUTH + LIMIT SYSTEM

Since you want SaaS:

- Limit:
 - Number of runs
 - Max runtime
 - Storage

BIG REALITY CHECK (VERY IMPORTANT)

Colab limitations

- No official automation API
- Sessions disconnect
- Rate limits
- Might violate ToS if abused

BETTER HYBRID IDEA (SMART MOVE)

Instead of ONLY Colab:

Support:

Source	Purpose
Colab	Free users

Source	Purpose
Local GPU	Advanced users
Cloud (AWS/GCP)	Paid users

ADVANCED FEATURES (MAKE THIS WORLD-CLASS)

1. AI Notebook Fixer

- Detect errors
- Auto-fix code using LLM

2. Training Visualizer

- Loss graphs
- Accuracy curves

3. Auto Retry System

- If runtime crashes → restart

4. Model Versioning

- v1, v2, v3 tracking

5. Hyperparameter Tuning Agent

- Run multiple versions automatically

6. Multi-Agent System (YOUR STYLE 😊)

- Agent 1: Data Loader
- Agent 2: Trainer
- Agent 3: Evaluator
- Agent 4: Optimizer

REAL PRODUCT NAME IDEAS

- ColabFlow
 - TrainHub AI
 - NotebookRunner
 - ModelForge
 - AutoTrain Cloud
-

STEP-BY-STEP BUILD PLAN (START SIMPLE)

Phase 1 (MVP)

- Upload `.ipynb`
 - Run via Playwright
 - Download model
-

Phase 2

- Job tracking
 - Logs
 - Storage
-

Phase 3

- Multi-user SaaS
 - Queue system
 - Limits
-

Phase 4

- AI automation
 - Multi-agent system
-

TECH STACK (MATCH YOUR SKILLS)

You should use:

Backend

- FastAPI (perfect for AI)

Frontend

- React (you already use)

Queue

- RabbitMQ / Redis

Automation

- Playwright (VERY IMPORTANT)

SIMPLE MVP ARCHITECTURE

```

React UI
  ↓
FastAPI
  ↓
Playwright Worker
  ↓
Google Colab
  ↓
Google Drive → Download → Store
  
```

HONEST FEEDBACK

This idea is:

- ✓ Very practical
- ✓ Can be built step-by-step
- ✓ Perfect for your AI portfolio
- ⚠ Needs careful handling of Colab limits

If you want next step

I can design:

- 🧠 Exact microservice architecture (deep level)
- ⚙️ Playwright automation script
- 🧩 Full backend API structure
- 🗄️ Storage system (models + logs)
- 🧠 AI agent integration (next level)

Just tell me:

👉 “go deeper” or “build MVP step-by-step”

Exact microservice architecture

Below is an exact **microservice architecture** for your application.

Your product is not just a file uploader. It is really a **distributed notebook-training orchestration platform**.

A good internal name for this system could be:

Notebook-to-Model Training Platform

or more technically:

Remote Notebook Training Orchestrator

1. Core product goal

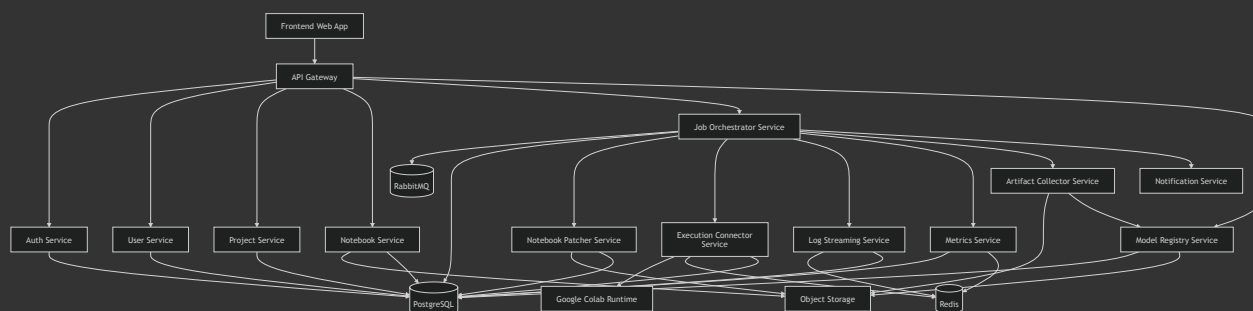
The system should let a user:

- upload a `.ipynb` notebook
- configure training inputs
- send that notebook to a Colab execution environment
- monitor execution
- collect logs, metrics, and artifacts
- save the resulting trained model in your platform
- download the trained model later

So the architecture must support:

- file ingestion
 - notebook parsing and patching
 - remote execution orchestration
 - job lifecycle management
 - artifact collection
 - storage
 - user/project/model management
 - optional auto-retry and notebook repair
-

2. Top-level architecture



This is the full service map. Now let's break it into exact services.

3. Exact microservices

A. API Gateway

This is the single public backend entry point.

Responsibilities

- route frontend requests to internal services
- validate auth token
- rate limiting
- request logging
- unify response format
- expose REST/GraphQL endpoints

Example endpoints

- `POST /auth/login`
- `POST /notebooks/upload`
- `POST /jobs/start`
- `GET /jobs/:id`
- `GET /jobs/:id/logs`
- `GET /models/:id/download`

Why separate it

This keeps the frontend simple and prevents direct exposure of internal services.

B. Auth Service

Handles identity and access.

Responsibilities

- signup/login
- JWT issuing
- refresh tokens
- password hashing
- roles and permissions

Roles

- user
- team-admin
- platform-admin

Database tables

- users
- refresh_tokens
- roles
- permissions
- user_sessions

Notes

For MVP this can be merged into API Gateway, but for real SaaS it should be separated.

C. User / Workspace Service

Handles user account space and ownership.

Responsibilities

- profile management
- teams/workspaces
- quotas
- subscription plan limits
- ownership of notebooks/projects/models/jobs

Stores

- workspaces
- members
- storage_usage

- run_usage
- plan_limits

Why important

Because later you may support:

- individuals
 - teams
 - enterprise accounts
-

D. Project Service

A project groups notebooks, jobs, models, datasets, and settings.

Responsibilities

- create project
- attach notebooks to project
- define environment settings
- default artifact storage settings
- version project metadata

Example

A user may have:

Project: Dog Breed Classifier

- notebook: `train.ipynb`
- dataset config
- model versions
- past runs

Tables

- projects
 - project_members
 - project_settings
 - project_tags
-

E. Notebook Service

This service receives, stores, validates, and versions notebook files.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Responsibilities

- upload `.ipynb`
- validate notebook format
- extract notebook metadata
- store original notebook
- maintain notebook versions
- mark notebook as ready for execution

Main flow

1. user uploads notebook
2. service validates JSON structure
3. notebook stored in object storage
4. notebook metadata saved in PostgreSQL
5. event emitted: `notebook.uploaded`

Metadata extracted

- notebook name
- kernelspec
- python version
- imported libraries
- presence of training loop
- save paths
- dataset mount logic
- suspicious cells

Tables

- notebooks
- notebook_versions
- notebook_metadata
- notebook_files

Object storage

Store:

- original notebook
- patched notebook
- execution-ready notebook

F. Notebook Analyzer Service

This service inspects notebook content semantically.

Responsibilities

- detect framework: PyTorch / TensorFlow / scikit-learn / XGBoost
- detect model save commands
- detect missing artifact export steps
- detect hardcoded paths
- detect runtime assumptions
- identify dataset source
- identify whether notebook is training/inference/evaluation only

Example detections

- `torch.save(model.state_dict(), 'model.pth')`
- `model.save('my_model.h5')`
- missing `drive.mount`
- notebook saves only to local `/content/`

Output

Structured notebook analysis JSON:

```
{
  "framework": "pytorch",
  "detected_artifacts": ["model.pth"],
  "needs_drive_mount": true,
  "needs_artifact_export_patch": true,
  "has_training_loop": true,
  "estimated_complexity": "medium"
}
```

Why separate it

Because analysis logic can become advanced later with LLM-based interpretation.

G. Notebook Patcher Service

This is one of the most important services.

It transforms the uploaded notebook into an execution-safe notebook for Colab.

Responsibilities

- insert initialization cells

- inject Google Drive mount logic
- inject artifact copy/export logic
- inject heartbeat logging
- inject structured metrics logging
- inject environment checks
- inject failure handling wrappers
- rewrite file paths if needed

Example patches

At notebook start:

```
import os, json, time
RUN_ID = "job_123"
ARTIFACT_DIR = "/content/platform_artifacts"
os.makedirs(ARTIFACT_DIR, exist_ok=True)
```

Mount logic:

```
from google.colab import drive
drive.mount('/content/drive')
```

Artifact export logic:

```
import shutil
shutil.copy("model.pth", "/content/drive/MyDrive/platform_runs/job_123/model.pth")
```

Metrics logger:

```
def log_metric(name, value, step):
    with open("/content/platform_artifacts/metrics.jsonl", "a") as f:
        f.write(json.dumps({"name": name, "value": value, "step": step}) + "\n")
```

Outputs

- patched notebook file
- patch manifest
- injection report

Patch manifest example

```
{
  "inserted_cells": 5,
  "rewritten_paths": 3,
  "artifact_exports": ["model.pth"],
  "logging_enabled": true
}
```

H. Job Orchestrator Service

This is the brain of the backend.

Responsibilities

- create training jobs
- manage job state transitions
- coordinate services
- enqueue work
- retry failed jobs
- maintain execution lifecycle
- decide which execution backend to use

State machine

A job may move through:

- `created`
- `queued`
- `preparing`
- `patched`
- `launching`
- `running`
- `collecting_artifacts`
- `completed`
- `failed`
- `cancelled`
- `timed_out`

What it does

When a user clicks "Train":

1. validate notebook version
2. create job record
3. send notebook to analyzer
4. send to patcher
5. enqueue execution task
6. wait for executor updates
7. stream status to frontend
8. trigger artifact collection
9. register model
10. notify user

Tables

- jobs
 - job_events
 - job_status_history
 - job_runtime_config
 - job_failures
-

I. Execution Connector Service

This service knows how to talk to the external execution environment.

For your case, this is the **Colab integration service**.

Responsibilities

- open Colab session
- upload patched notebook
- start execution
- monitor runtime state
- collect execution progress signals
- detect notebook completion
- recover from session interruptions

Important reality

Because Colab does not give you a clean official orchestration API for this exact workflow, this service will likely use:

- browser automation with Playwright
- authenticated browser profile/session
- DOM/event monitoring
- file upload automation
- notebook execution triggering
- artifact pulling through Google Drive or browser download

Internal modules

- session manager
- browser automation worker
- credential manager
- runtime detector
- completion detector

State examples

- `browser_booting`
- `opening_colab`
- `uploading_notebook`
- `waiting_runtime`
- `executing_cells`
- `watching_progress`
- `job_done`
- `session_failed`

Why this must be isolated

Because this is the most fragile part of the system. If Colab UI changes, only this service should break, not your whole platform.

J. Runtime Session Service

This service tracks active external runtime sessions.

Responsibilities

- session leasing
- mapping jobs to runtime sessions
- session timeout monitoring
- cleanup and release
- track browser profiles / automation contexts

Stores

- `active_sessions`
- `session_heartbeats`
- `last_seen`
- `session_errors`
- `session_resource_type`

Why useful

You do not want two jobs accidentally sharing the same remote Colab runtime unless intentionally designed.

K. Log Streaming Service

This handles live training logs.

Responsibilities

- capture stdout/stderr streams
- capture notebook cell output
- stream logs to UI via WebSocket / SSE
- persist logs for historical view

Data sources

- injected notebook log files
- browser automation output capture
- Colab cell output polling
- execution service event logs

Storage pattern

- hot logs in Redis
- permanent logs in PostgreSQL or object storage

Endpoints

- GET `/jobs/:id/logs`
- WebSocket `/ws/jobs/:id/logs`

UI usage

This powers the “live terminal” experience.

L. Metrics Service

This service collects structured training metrics.

Responsibilities

- parse metrics.jsonl
- store epoch/step/loss/accuracy
- generate chart-friendly timeseries
- expose metrics to frontend

Example metrics

- training_loss
- validation_loss

- accuracy
- learning_rate
- GPU memory
- epoch time

Tables

- job_metrics
- metric_series
- final_metrics_summary

UI output

You can show:

- loss curve
 - accuracy curve
 - epoch table
 - best checkpoint
-

M. Artifact Collector Service

After training finishes, this service gathers outputs.

Responsibilities

- fetch model files
- fetch checkpoints
- fetch evaluation reports
- fetch confusion matrices
- fetch logs
- validate artifact integrity
- move artifacts to permanent storage

Artifact examples

- `model.pth`
- `best_model.ckpt`
- `tokenizer.json`
- `label_map.json`
- `metrics.json`

- `train.log`

Storage flow

1. artifact found in Colab/Drive
2. downloaded to temporary worker space
3. checksum computed
4. uploaded to object storage
5. metadata stored in DB

Tables

- artifacts
- artifact_versions
- artifact_checksums
- artifact_download_events

N. Model Registry Service

This service turns raw artifacts into managed models.

Responsibilities

- register trained models
- assign version numbers
- connect model to project/job/notebook
- store model metadata
- expose download links
- mark best/latest/stable versions

Example

Project: **Dog Breed Classifier**

Model registry:

- v1 trained from notebook A
- v2 trained from notebook A updated
- v3 trained with improved hyperparameters

Stores metadata

- framework
- model format

- size
- checksum
- training job
- source notebook version
- metrics summary
- tags

Tables

- models
- model_versions
- model_files
- model_tags
- model_lineage

Extra power

Later this service can support:

- deployable models
- model cards
- approval workflow
- rollback

O. Storage Service

This is usually not a custom-coded service at first, but logically it is a platform component.

Responsibilities

- store notebook files
- store patched notebooks
- store model files
- store logs
- store metrics exports
- store temp artifacts

Recommended implementation

- S3-compatible object storage
- or MinIO for self-hosted development

Bucket examples

- notebooks-original
 - notebooks-patched
 - job-artifacts
 - models
 - logs
-

P. Notification Service

This informs the user of system events.

Responsibilities

- job started
- job failed
- job completed
- model ready for download
- quota exceeded
- retry scheduled

Channels

- in-app notifications
 - email
 - webhook
 - later maybe Telegram/Discord
-

Q. Billing / Quota Service

For SaaS, this becomes essential.

Responsibilities

- enforce storage quota
- run count quota
- concurrent job quota
- team plan limits
- usage metering

Example limits

Free plan:

- 3 runs/day
- 1 active job
- 2 GB model storage

Pro plan:

- 50 runs/day
 - 5 active jobs
 - 50 GB storage
-

4. Message queue architecture

You should not make services call each other synchronously for everything.

Use a queue for long-running tasks.

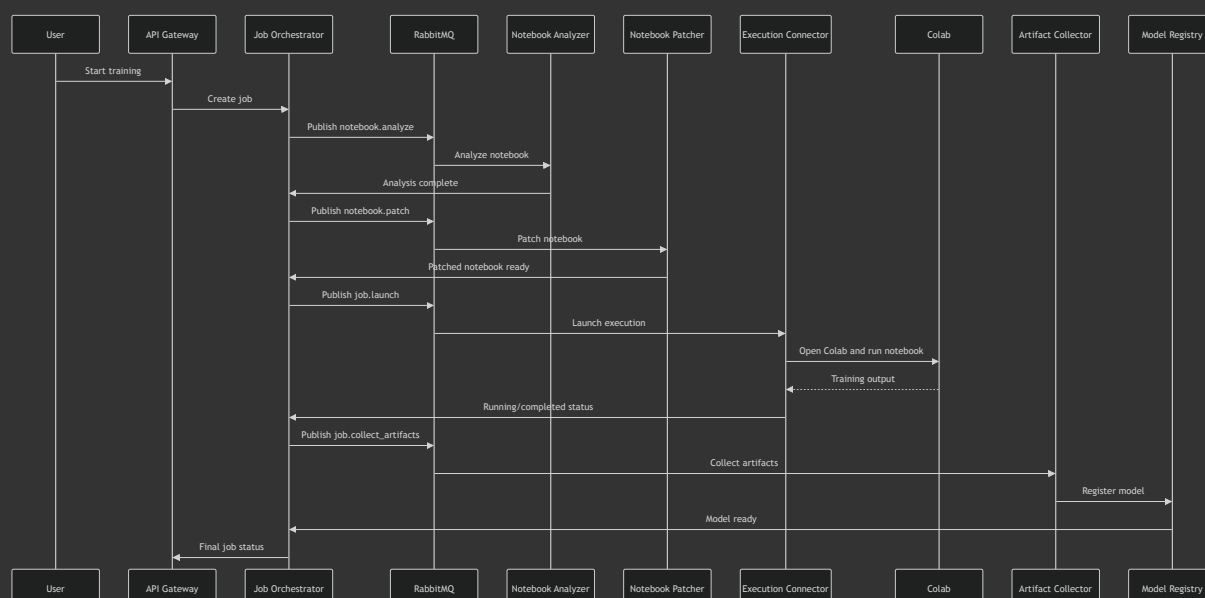
Recommended

- RabbitMQ for event/task routing
- Redis for caching and lightweight state

Queue topics

- `notebook.uploaded`
- `notebook.analyze`
- `notebook.patch`
- `job.launch`
- `job.status.update`
- `job.logs.ingest`
- `job.metrics.ingest`
- `job.collect_artifacts`
- `model.register`
- `notification.send`

Example async flow



5. Exact database split

Use **PostgreSQL** as the main metadata database.

Shared relational metadata

Good for:

- users
- jobs
- project ownership
- model versions
- notebook metadata
- status history

Redis

Good for:

- active job state
- websocket log fanout
- temporary session tokens
- short-lived execution status

Object storage

Good for:

- .ipynb files
 - model binaries
 - logs
 - checkpoints
 - exported metrics files
-

6. Service-by-service internal database ownership

A cleaner microservice design is to let each service own its core tables.

Auth Service owns

- users
- credentials
- refresh_tokens
- sessions

Project Service owns

- projects
- workspace_members
- project_settings

Notebook Service owns

- notebooks
- notebook_versions
- notebook_files
- notebook_metadata

Job Orchestrator owns

- jobs
- job_events
- job_status_history
- runtime_config
- retry_records

Model Registry owns

- models
- model_versions
- model_artifacts
- lineage

Notification Service owns

- notifications
- notification_delivery_attempts

For MVP, all these tables can still live in one PostgreSQL database, but ownership should be logically separated.

7. Exact execution architecture for Colab

This is the hardest part, so it needs internal modular design.

Execution Connector internal modules

1. Colab Session Manager

- launches authenticated browser context
- ensures Google login session is active
- opens Colab page
- reserves session for one job

2. Notebook Upload Controller

- uploads the patched `.ipynb`
- confirms notebook loaded correctly
- waits for UI ready state

3. Runtime Allocator

- connects to GPU/TPU runtime if needed
- checks runtime availability
- detects disconnect/reconnect

4. Run Controller

- clicks Run All
- responds to notebook warnings
- monitors active cell execution

5. Output Scraper

- reads cell outputs
- captures progress text
- extracts visible errors

6. Completion Detector

- identifies run completion
- determines success/failure
- confirms artifact export has completed

7. Download Bridge

- pulls artifacts from mounted Drive or download UI
 - sends artifact paths to Artifact Collector
-

8. Recommended deployment architecture

For actual deployment, use containerized services.

Suggested runtime

- Docker
- Kubernetes later
- Docker Compose for local dev

Minimal deployable layout

- frontend
- api-gateway
- auth-service
- notebook-service
- notebook-analyzer
- notebook-patcher
- job-orchestrator
- execution-connector
- log-service
- metrics-service
- artifact-collector
- model-registry

- notification-service
 - postgres
 - redis
 - rabbitmq
 - minio
-

9. Recommended language per service

Because you work well with AI tooling, I would recommend this split:

Python services

Use Python for:

- notebook analyzer
- notebook patcher
- job orchestrator
- metrics parser
- artifact collector
- model registry
- execution automation if Playwright Python is used

Why:

- easier `.ipynb` parsing
- strong ML ecosystem
- easy artifact handling

Node.js/TypeScript services

Use Node only if you want for:

- API Gateway
- real-time websocket log server
- frontend backend adapter

But honestly, for this project, **FastAPI + Python across most services** is cleaner.

10. Exact request flow from upload to model download

Flow 1: notebook upload

1. frontend uploads `.ipynb`
2. API Gateway forwards to Notebook Service
3. Notebook Service validates and stores original file
4. metadata saved
5. event `notebook.uploaded`

Flow 2: create training job

1. user selects notebook and clicks train
2. Job Orchestrator creates job record
3. notebook analysis requested
4. patching requested
5. patched notebook stored
6. execution task queued

Flow 3: remote execution

1. Execution Connector opens Colab
2. uploads patched notebook
3. runtime attached
4. notebook runs
5. logs and metrics streamed
6. job state updated live

Flow 4: artifact capture

1. training ends
2. Artifact Collector fetches outputs
3. checksums and metadata created
4. files uploaded to object storage

Flow 5: model registration

1. Model Registry creates model version
2. links it to project/job/notebook
3. generates download endpoint
4. user sees model in dashboard

11. Failure handling architecture

This product absolutely needs robust failure design.

Common failure cases

- notebook invalid
- missing dependencies
- Colab runtime unavailable
- runtime disconnected
- browser automation failed
- artifact missing
- model file path not detected
- user quota exceeded

Required failure modules

Retry policy

- retry browser launch
- retry notebook upload
- retry artifact fetch
- do not blindly rerun long training without user approval

Dead letter queue

Failed jobs/events go here for inspection.

Failure classifier

Classify errors into:

- user code error
- platform patch error
- execution environment failure
- external service failure

Recovery state

A job should not just say “failed”.

It should say:

- failed during patching
- failed during runtime allocation
- failed during artifact collection

That makes your platform look professional.

12. Recommended observability stack

You need internal observability too.

Add:

- centralized logs
- job traces
- service metrics
- alerting

Track:

- queue length
- average job duration
- success rate
- artifact collection failure rate
- Colab launch failure rate
- patch success rate

Later use:

- Prometheus
 - Grafana
 - OpenTelemetry
 - Loki
-

13. Best MVP microservice cut

Do not start with 15 separate deployables.

Start with a practical cut.

MVP services

1. Frontend

React app

2. API Service

Handles:

- auth
- users
- projects
- notebook metadata
- model listing

3. Notebook Worker Service

Handles:

- notebook analysis
- patching

4. Job Worker Service

Handles:

- job orchestration
- queue consumption
- status transitions

5. Execution Worker Service

Handles:

- Colab automation
- runtime monitoring
- log extraction

6. Artifact/Model Service

Handles:

- artifact collection
- model registration
- download serving

Infra

- PostgreSQL
- Redis
- RabbitMQ
- S3/MinIO

This is much easier to build.

14. Scaled production microservice cut

Once MVP works, split into:

- API Gateway
- Auth Service
- User/Workspace Service
- Project Service
- Notebook Service
- Notebook Analyzer Service
- Notebook Patcher Service
- Job Orchestrator Service
- Runtime Session Service
- Execution Connector Service
- Log Streaming Service
- Metrics Service
- Artifact Collector Service
- Model Registry Service
- Notification Service
- Billing/Quota Service

That is your true production architecture.

15. Exact recommended folder/service structure

If you use a monorepo:

```
platform/
├── frontend/
├── services/
│   ├── api-gateway/
│   ├── auth-service/
│   ├── project-service/
│   ├── notebook-service/
│   ├── notebook-analyzer/
│   ├── notebook-patcher/
│   ├── job-orchestrator/
│   ├── execution-connector/
│   ├── runtime-session-service/
│   ├── log-streaming-service/
│   ├── metrics-service/
│   ├── artifact-collector/
│   ├── model-registry/
│   ├── notification-service/
│   └── billing-service/
├── packages/
│   ├── shared-schemas/
│   ├── shared-events/
│   ├── shared-auth/
│   └── shared-utils/
├── infra/
│   ├── docker/
│   ├── k8s/
│   ├── terraform/
│   └── compose/
└── docs/
```

16. Best internal event contracts

Use event-driven communication for long-running tasks.

Example events

notebook uploaded

```
{
  "event": "notebook.uploaded",
  "notebook_id": "nb_001",
  "project_id": "proj_001",
  "user_id": "usr_001",
  "version_id": "v3",
  "file_url": "s3://notebooks/original/nb_001_v3.ipynb",
  "created_at": "2026-03-24T10:00:00Z"
}
```

job created

```
{
  "event": "job.created",
  "job_id": "job_123",
  "notebook_id": "nb_001",
  "notebook_version_id": "v3",
  "execution_backend": "colab",
  "requested_gpu": true
}
```

model registered

```
{
  "event": "model.registered",
  "model_id": "mdl_101",
  "job_id": "job_123",
  "project_id": "proj_001",
  "artifact_url": "s3://models/mdl_101/model.pth",
  "framework": "pytorch",
  "version": "1.0.0"
}
```

17. Security architecture

This matters a lot because notebooks can execute arbitrary code.

You must design for:

- sandboxing workers
- isolating temp files
- restricted credential access
- per-job workspace separation
- audit logs
- virus/malware scanning of uploads
- notebook trust model
- model download authorization

Critical rule

Never let an uploaded notebook directly access your main backend secrets.

The execution worker should only get:

- job-scoped temp credentials
 - restricted storage tokens
 - short-lived artifact upload credentials
-

18. My exact recommendation for your build

For your skillset and your product direction, I recommend this architecture:

Stack

- **Frontend:** React
- **API + most services:** FastAPI
- **Queue:** RabbitMQ
- **Cache/live state:** Redis

- **DB:** PostgreSQL
- **Object storage:** MinIO first, S3 later
- **Automation:** Playwright
- **Containerization:** Docker Compose first

MVP microservices

- API Service
- Notebook Worker
- Job Orchestrator Worker
- Execution Connector Worker
- Artifact Registry Service

That is the sweet spot.

19. Final architecture summary in one sentence

Your application should be designed as an **event-driven notebook orchestration platform** where uploaded notebooks are analyzed, patched, executed in Colab through a dedicated execution connector, and then all resulting artifacts are collected, versioned, and exposed through a model registry for download.

If you want, the next best step is: **I can design the exact database schema for all these microservices with tables, fields, and relationships.**

yes do that

Perfect — now we go **deep into database design (production-level)** 🔥

This is the **core foundation** of your entire platform.

I'll design this like a real SaaS used by thousands of users.

DATABASE ARCHITECTURE OVERVIEW

We will use:

- **PostgreSQL** → relational metadata
 - **Redis** → runtime + cache
 - **Object Storage (S3/MinIO)** → files
-

DATABASE LAYERS

1. Core Metadata (PostgreSQL)

Stores:

- users
- projects
- notebooks
- jobs
- models
- artifacts

2. Runtime State (Redis)

Stores:

- active job status
- logs (temporary)
- session states

3. File Storage (S3/MinIO)

Stores:

- .ipynb
- models (.pt, .h5)
- logs
- checkpoints

FULL DATABASE SCHEMA (PRODUCTION LEVEL)

1. USERS & AUTH

users

```
CREATE TABLE users (
  id UUID PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  full_name TEXT,
  avatar_url TEXT,
  role TEXT DEFAULT 'user',
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP
);
```

sessions

```
CREATE TABLE sessions (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  refresh_token TEXT,
  expires_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);
```

api_keys (for future integrations)

```
CREATE TABLE api_keys (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  key_hash TEXT,
  name TEXT,
  created_at TIMESTAMP DEFAULT NOW(),
  revoked BOOLEAN DEFAULT FALSE
);
```

2. WORKSPACES / PROJECTS

workspaces

```
CREATE TABLE workspaces (
  id UUID PRIMARY KEY,
  name TEXT,
  owner_id UUID REFERENCES users(id),
  plan TEXT DEFAULT 'free',
  created_at TIMESTAMP DEFAULT NOW()
);
```

workspace_members

```
CREATE TABLE workspace_members (
  id UUID PRIMARY KEY,
  workspace_id UUID REFERENCES workspaces(id),
  user_id UUID REFERENCES users(id),
  role TEXT,
  joined_at TIMESTAMP DEFAULT NOW()
);
```

projects

```
CREATE TABLE projects (
  id UUID PRIMARY KEY,
  workspace_id UUID REFERENCES workspaces(id),
  name TEXT,
  description TEXT,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW()
);
```

3. NOTEBOOK SYSTEM

notebooks

```
CREATE TABLE notebooks (
  id UUID PRIMARY KEY,
  project_id UUID REFERENCES projects(id),
  name TEXT,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW()
);
```

notebook_versions

```
CREATE TABLE notebook_versions (
  id UUID PRIMARY KEY,
  notebook_id UUID REFERENCES notebooks(id),
  version_number INT,
  file_url TEXT, -- S3 path
  patched_file_url TEXT,
  size_bytes BIGINT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

notebook_metadata

```
CREATE TABLE notebook_metadata (
  id UUID PRIMARY KEY,
  notebook_version_id UUID REFERENCES notebook_versions(id),
  framework TEXT, -- pytorch/tensorflow
  has_training_loop BOOLEAN,
  detected_artifacts JSONB,
  dependencies JSONB,
  analysis JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);
```

4. JOB SYSTEM (MOST IMPORTANT)

jobs

```
CREATE TABLE jobs (
  id UUID PRIMARY KEY,
  project_id UUID REFERENCES projects(id),
  notebook_version_id UUID REFERENCES notebook_versions(id),
  user_id UUID REFERENCES users(id),
  status TEXT,
  execution_backend TEXT, -- colab/aws/local
  requested_gpu BOOLEAN DEFAULT FALSE,
  started_at TIMESTAMP,
  completed_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);
```

job_status_history


```
CREATE TABLE job_status_history (
  id UUID PRIMARY KEY,
  job_id UUID REFERENCES jobs(id),
  status TEXT,
  message TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

job_config

```
CREATE TABLE job_config (
  id UUID PRIMARY KEY,
  job_id UUID REFERENCES jobs(id),
  parameters JSONB,
  environment JSONB,
  timeout_seconds INT,
  retry_count INT DEFAULT 0
);
```

job_failures

```
CREATE TABLE job_failures (
  id UUID PRIMARY KEY,
  job_id UUID REFERENCES jobs(id),
  error_type TEXT,
  error_message TEXT,
  stack_trace TEXT,
  failed_at TIMESTAMP DEFAULT NOW()
);
```



5. LOGS & METRICS

job_logs

```
CREATE TABLE job_logs (
  id UUID PRIMARY KEY,
  job_id UUID REFERENCES jobs(id),
  log_line TEXT,
  log_level TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

job_metrics

```
CREATE TABLE job_metrics (
  id UUID PRIMARY KEY,
  job_id UUID REFERENCES jobs(id),
  metric_name TEXT,
  metric_value FLOAT,
  step INT,
  recorded_at TIMESTAMP DEFAULT NOW()
);
```

job_summary

```
CREATE TABLE job_summary (
  job_id UUID PRIMARY KEY,
  final_accuracy FLOAT,
  final_loss FLOAT,
  best_epoch INT,
  total_epochs INT
);
```

6. ARTIFACT SYSTEM

artifacts

```
CREATE TABLE artifacts (
  id UUID PRIMARY KEY,
  job_id UUID REFERENCES jobs(id),
  name TEXT,
  type TEXT, -- model/checkpoint/log
  file_url TEXT,
  size_bytes BIGINT,
  checksum TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

artifact_versions

```
CREATE TABLE artifact_versions (
  id UUID PRIMARY KEY,
  artifact_id UUID REFERENCES artifacts(id),
  version INT,
  file_url TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

7. MODEL REGISTRY (VERY IMPORTANT)

models

```
CREATE TABLE models (
  id UUID PRIMARY KEY,
  project_id UUID REFERENCES projects(id),
  name TEXT,
  framework TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

model_versions

```
CREATE TABLE model_versions (
  id UUID PRIMARY KEY,
  model_id UUID REFERENCES models(id),
  job_id UUID REFERENCES jobs(id),
  version TEXT,
  artifact_id UUID REFERENCES artifacts(id),
  accuracy FLOAT,
  is_best BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW()
);
```

model_tags

```
CREATE TABLE model_tags (
  id UUID PRIMARY KEY,
  model_version_id UUID REFERENCES model_versions(id),
  tag TEXT
);
```

model_lineage (ADVANCED 🔥)

```
CREATE TABLE model_lineage (
  id UUID PRIMARY KEY,
  parent_model_version_id UUID,
  child_model_version_id UUID,
  relation TEXT -- fine-tuned / retrained
);
```

🔔 8. NOTIFICATIONS

notifications

```
CREATE TABLE notifications (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  title TEXT,
  message TEXT,
  type TEXT,
  read BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW()
);
```

💰 9. QUOTA & BILLING

usage_tracking

```
CREATE TABLE usage_tracking (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  jobs_run INT DEFAULT 0,
  storage_used BIGINT DEFAULT 0,
  last_updated TIMESTAMP DEFAULT NOW()
);
```

plans

```
CREATE TABLE plans (
  id UUID PRIMARY KEY,
  name TEXT,
  max_jobs_per_day INT,
  max_storage BIGINT,
  max_concurrent_jobs INT
);
```

⚡ REDIS STRUCTURE (IMPORTANT)

Use Redis for real-time:

Active job state

```
job:123 {  
  status: "running",  
  progress: 45,  
  current_epoch: 3,  
  eta: 120  
}
```

Live logs stream

logs:job:123 → stream of log lines

WebSocket channels

ws:job:123

OBJECT STORAGE STRUCTURE

```
/notebooks/  
  original/{id}.ipynb  
  patched/{id}.ipynb  
  
/jobs/  
  {job_id}/  
    logs/  
    metrics.json  
    artifacts/  
  
/models/  
  {model_id}/  
    v1/  
    v2/
```

RELATIONSHIP SUMMARY

CORE RELATION FLOW

```
User  
  → Workspace  
    → Project  
      → Notebook  
        → Notebook Version  
          → Job  
            → Artifacts  
            → Model Version
```

WHY THIS DESIGN IS POWERFUL

This schema allows you to:

- ✓ Version notebooks
- ✓ Track every training run
- ✓ Store multiple models
- ✓ Compare models
- ✓ Reproduce experiments
- ✓ Scale to teams
- ✓ Add AI agents later
- ✓ Add hyperparameter tuning
- ✓ Add multi-cloud execution

NEXT STEP (VERY IMPORTANT)

You now have:

- ✓ Full microservice architecture
- ✓ Full database schema

Next best step I recommend:

I can design:

Option A

Exact backend API structure (routes + request/response + validation)

Option B

Playwright Colab automation (real working script)

Option C

Job execution engine code architecture (queue + workers)

Option D

Frontend UI system (React structure + pages + components)

Just tell me:

-  "build backend API"
-  "build execution engine"
-  "build frontend system"

do that one by one I can design:

Option A

Exact backend API structure (routes + request/response + validation)

🔥 Option B

Playwright Colab automation (real working script)

🔥 Option C

Job execution engine code architecture (queue + workers)

🔥 Option D

Frontend UI system (React structure + pages + components)

Great. I'll do them in order:

1. **Backend API structure**
2. **Playwright Colab automation**
3. **Job execution engine**
4. **Frontend UI system**

I'll design them so they fit together as one real product.

Option A — Exact backend API structure

This API should sit behind your API gateway and expose a clean SaaS-style contract to the frontend.

1. API modules

Split routes into these domains:

- auth
- workspaces
- projects
- notebooks
- jobs
- logs
- metrics
- artifacts
- models
- notifications
- quotas/admin

2. Route map

Auth routes

POST /api/v1/auth/register

Creates a user account.

Request

```
{
  "email": "user@example.com",
  "password": "StrongPassword123",
  "full_name": "Dasun Charuka"
}
```

Validation

- email must be valid
- password min length 8
- full_name optional but max 120 chars

Response

```
{
  "user": {
    "id": "usr_123",
    "email": "user@example.com",
    "full_name": "Dasun Charuka"
  },
  "tokens": {
    "access_token": "jwt_access",
    "refresh_token": "jwt_refresh"
  }
}
```

POST /api/v1/auth/login

Request

```
{
  "email": "user@example.com",
  "password": "StrongPassword123"
}
```

Response

```
{
  "user": {
    "id": "usr_123",
    "email": "user@example.com",
    "full_name": "Dasun Charuka"
  },
  "tokens": {
    "access_token": "jwt_access",
    "refresh_token": "jwt_refresh"
  }
}
```

POST /api/v1/auth/refresh

Request

```
{
  "refresh_token": "jwt_refresh"
}
```

Response

```
{
  "access_token": "new_access_token"
}
```

GET /api/v1/auth/me

Response

```
{
  "id": "usr_123",
  "email": "user@example.com",
  "full_name": "Dasun Charuka",
  "role": "user"
}
```

Workspace routes

POST /api/v1/workspaces

Request

```
{
  "name": "My AI Workspace"
}
```

Validation

- name required
- max length 100

Response

```
{
  "id": "ws_001",
  "name": "My AI Workspace",
  "plan": "free",
  "owner_id": "usr_123"
}
```

GET /api/v1/workspaces

Returns all workspaces user belongs to.

GET /api/v1/workspaces/{workspace_id}

Returns single workspace details.

POST /api/v1/workspaces/{workspace_id}/members

Adds team members later.

Request

```
{
  "email": "member@example.com",
  "role": "editor"
}
```

Project routes

POST /api/v1/projects

Request

```
{
  "workspace_id": "ws_001",
  "name": "Dog Breed Classifier",
  "description": "Training image classifier from notebooks"
}
```

Response

```
{
  "id": "proj_001",
  "workspace_id": "ws_001",
  "name": "Dog Breed Classifier",
  "description": "Training image classifier from notebooks"
}
```

GET /api/v1/projects

Supports filters:

- workspace_id
- search
- page
- page_size

Example

GET /api/v1/projects?workspace_id=ws_001&page=1&page_size=20

GET /api/v1/projects/{project_id}

Returns project detail plus summary counts.

Response

```
{
  "id": "proj_001",
  "name": "Dog Breed Classifier",
  "description": "Training image classifier from notebooks",
  "stats": {
    "notebooks_count": 3,
    "jobs_count": 12,
    "models_count": 5
  }
}
```

PATCH /api/v1/projects/{project_id}

Request

```
{
  "name": "Updated Project Name",
  "description": "Updated description"
}
```

DELETE /api/v1/projects/{project_id}

Soft delete recommended.

Notebook routes

POST /api/v1/notebooks/upload

Use multipart upload.

Form fields

- project_id
- file (.ipynb)

Validation

- file required
- extension must be .ipynb
- size within plan limit
- valid JSON notebook structure

Response

```
{
  "notebook": {
    "id": "nb_001",
    "project_id": "proj_001",
    "name": "train_model.ipynb",
    "latest_version_id": "nbv_001"
  }
}
```

GET /api/v1/notebooks/{notebook_id}

Response

```
{
  "id": "nb_001",
  "project_id": "proj_001",
  "name": "train_model.ipynb",
  "created_at": "2026-03-24T10:00:00Z",
  "latest_version": {
    "id": "nbv_001",
    "version_number": 1,
    "size_bytes": 124533
  }
}
```

GET /api/v1/notebooks/{notebook_id}/versions

Response

```
{
  "items": [
    {
      "id": "nbv_001",
      "version_number": 1,
      "created_at": "2026-03-24T10:00:00Z"
    }
  ]
}
```

GET /api/v1/notebooks/versions/{version_id}/analysis

Returns analyzer output.

Response

```
{
  "framework": "pytorch",
  "has_training_loop": true,
  "detected_artifacts": [
    "model.pth"
  ],
  "dependencies": [
    "torch",
    "numpy",
    "sklearn"
  ],
  "needs_drive_mount": true,
  "needs_artifact_export_patch": true
}
```

POST /api/v1/notebooks/versions/{version_id}/patch

Triggers patch generation manually or for preview.

Request

```
{
  "artifact_paths": ["model.pth"],
  "enable_metrics_logging": true,
  "enable_drive_mount": true
}
```

Response

```
{
  "patched_version_id": "nbv_patch_001",
  "status": "patched"
}
```

GET /api/v1/notebooks/versions/{version_id}/download

Downloads original notebook.

GET /api/v1/notebooks/versions/{version_id}/patched-download

Downloads patched notebook.

Job routes

This is the most important API group.

POST /api/v1/jobs

Creates a training job.

Request

```
{
  "project_id": "proj_001",
  "notebook_version_id": "nbv_001",
  "execution_backend": "colab",
  "requested_gpu": true,
  "parameters": {
    "epochs": 10,
    "batch_size": 32
  },
  "environment": {
    "python_version": "3.10"
  },
  "artifact_paths": [
    "model.pth",
    "metrics.json"
  ],
  "timeout_seconds": 14400
}
```

Validation

- notebook_version_id required

- project must belong to user workspace
- backend must be one of `colab`, `local`, `cloud`
- timeout within plan limits
- artifact_paths array max length limit

Response

```
{
  "id": "job_001",
  "status": "created",
  "execution_backend": "colab",
  "requested_gpu": true,
  "created_at": "2026-03-24T10:05:00Z"
}
```

GET /api/v1/jobs

Filters:

- project_id
 - status
 - notebook_id
 - page
 - page_size
-

GET /api/v1/jobs/{job_id}

Response

```
{
  "id": "job_001",
  "project_id": "proj_001",
  "notebook_version_id": "nbv_001",
  "status": "running",
  "execution_backend": "colab",
  "requested_gpu": true,
  "started_at": "2026-03-24T10:06:00Z",
  "completed_at": null,
  "current_stage": "executing_cells",
  "progress": {
    "percent": 43,
    "message": "Epoch 3/10"
  }
}
```

GET /api/v1/jobs/{job_id}/history

Returns lifecycle history.

Response

```
{
  "items": [
    {
      "status": "created",
      "message": "Job created",
      "created_at": "2026-03-24T10:05:00Z"
    },
    {
      "status": "queued",
      "message": "Waiting for worker",
      "created_at": "2026-03-24T10:05:03Z"
    },
    {
      "status": "running",
      "message": "Notebook execution started",
      "created_at": "2026-03-24T10:06:00Z"
    }
  ]
}
```

POST /api/v1/jobs/{job_id}/cancel

Response

```
{
  "id": "job_001",
  "status": "cancel_requested"
}
```

POST /api/v1/jobs/{job_id}/retry

Response

```
{
  "id": "job_002",
  "retry_of_job_id": "job_001",
  "status": "created"
}
```

Log routes

GET /api/v1/jobs/{job_id}/logs

Query:

- cursor
- limit

Response

```
{
  "items": [
    {
      "timestamp": "2026-03-24T10:07:00Z",
      "level": "INFO",
      "line": "Epoch 1/10 - loss=0.8231"
    }
  ],
  "next_cursor": "abc123"
}
```

GET /api/v1/jobs/{job_id}/logs/stream

WebSocket or SSE stream.

SSE event format

```
{
  "timestamp": "2026-03-24T10:07:00Z",
  "level": "INFO",
  "line": "Epoch 1/10 - loss=0.8231"
}
```

Metrics routes

GET /api/v1/jobs/{job_id}/metrics

Response

```
{
  "series": [
    {
      "metric_name": "train_loss",
      "points": [
        { "step": 1, "value": 0.92 },
        { "step": 2, "value": 0.81 }
      ]
    },
    {
      "metric_name": "val_accuracy",
      "points": [
        { "step": 1, "value": 0.71 },
        { "step": 2, "value": 0.79 }
      ]
    }
  ]
}
```

GET /api/v1/jobs/{job_id}/summary

Response

```
{
  "final_accuracy": 0.89,
  "final_loss": 0.21,
  "best_epoch": 8,
  "total_epochs": 10
}
```

Artifact routes

GET /api/v1/jobs/{job_id}/artifacts

Response

```
{
  "items": [
    {
      "id": "art_001",
      "name": "model.pth",
      "type": "model",
      "size_bytes": 48582912
    },
    {
      "id": "art_002",
      "name": "metrics.json",
      "type": "metrics",
      "size_bytes": 2813
    }
  ]
}
```

GET /api/v1/artifacts/{artifact_id}

Returns metadata.

GET /api/v1/artifacts/{artifact_id}/download

Downloads artifact binary.

Model registry routes

POST /api/v1/models/register

Usually internal, but can also be user-triggered.

Request

```
{
  "project_id": "proj_001",
  "job_id": "job_001",
  "artifact_id": "art_001",
  "name": "dog-breed-model",
  "framework": "pytorch",
  "version": "v1"
}
```

Response

```
{
  "id": "mdl_001",
  "name": "dog-breed-model",
  "version_id": "mdl_v_001"
}
```

GET /api/v1/models

Filters:

- project_id
- search

- framework

GET /api/v1/models/{model_id}

Response

```
{
  "id": "mdl_001",
  "name": "dog-breed-model",
  "framework": "pytorch",
  "versions": [
    {
      "id": "mdl_v_001",
      "version": "v1",
      "accuracy": 0.89,
      "is_best": true
    }
  ]
}
```

GET /api/v1/model-versions/{model_version_id}/download

Downloads actual model file.

POST /api/v1/model-versions/{model_version_id}/mark-best

Marks version as best.

Notifications routes

GET /api/v1/notifications

POST /api/v1/notifications/{notification_id}/read

Quota routes

GET /api/v1/usage/me

Response

```
{
  "plan": "free",
  "jobs_run_today": 2,
  "max_jobs_per_day": 3,
  "storage_used_bytes": 104857600,
  "max_storage_bytes": 2147483648,
  "active_jobs": 1,
  "max_concurrent_jobs": 1
}
```

Admin routes

GET /api/v1/admin/jobs

GET /api/v1/admin/users

POST /api/v1/admin/jobs/{job_id}/force-fail

These are platform-admin only.

3. Standard response envelope

Use one clean format everywhere.

Success

```
{
  "success": true,
  "data": {}
}
```

Error

```
{
  "success": false,
  "error": {
    "code": "NOTEBOOK_INVALID",
    "message": "Uploaded notebook is not a valid .ipynb file"
  }
}
```

4. Error codes

Use stable machine-readable codes:

- AUTH_INVALID_CREDENTIALS
 - AUTH_TOKEN_EXPIRED
 - WORKSPACE_NOT_FOUND
 - PROJECT_NOT_FOUND
 - NOTEBOOK_INVALID
 - NOTEBOOK_TOO_LARGE
 - JOB_NOT_FOUND
 - JOB_ALREADY_FINISHED
 - JOB_QUOTA_EXCEEDED
 - ARTIFACT_NOT_FOUND
 - MODEL_NOT_FOUND
 - PERMISSION_DENIED
 - INTERNAL_ERROR
-

5. FastAPI schema example

```
from pydantic import BaseModel, Field, EmailStr
from typing import Optional, Literal

class RegisterRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=8, max_length=128)
    full_name: Optional[str] = Field(default=None, max_length=120)

class JobCreateRequest(BaseModel):
    project_id: str
    notebook_version_id: str
    execution_backend: Literal["colab", "local", "cloud"]
    requested_gpu: bool = False
    timeout_seconds: int = Field(default=14400, ge=60, le=86400)
    parameters: dict = {}
    environment: dict = {}
    artifact_paths: list[str] = []
```

6. Backend service structure

```
backend/
├── app/
│   ├── main.py
│   └── api/
│       ├── auth.py
│       ├── workspaces.py
│       ├── projects.py
│       ├── notebooks.py
│       ├── jobs.py
│       ├── logs.py
│       ├── metrics.py
│       ├── artifacts.py
│       ├── models.py
│       └── admin.py
├── schemas/
├── services/
├── repositories/
├── models/
├── workers/
└── core/
```

Option B — Playwright Colab automation

This is the fragile but exciting part.

Important: build this as a dedicated worker, not inside your API server.

1. Colab automation goal

The worker should:

- open Google Colab
- authenticate with a prepared Google session
- upload notebook
- connect runtime
- run all cells

- observe outputs
 - wait for artifacts to appear
 - download or sync them back
-

2. Recommended architecture for automation worker

```
execution_worker/  
├── runner.py  
├── colab_session.py  
├── notebook_uploader.py  
├── runtime_manager.py  
├── output_monitor.py  
├── artifact_bridge.py  
├── selectors.py  
└── config.py
```

3. Main automation flow

1. load browser profile with logged-in Google account
 2. open Colab
 3. upload patched notebook
 4. wait until notebook UI is ready
 5. switch runtime if needed
 6. click "Run all"
 7. handle confirmation dialogs
 8. monitor progress and failures
 9. confirm artifact export
 10. close session
-

4. Playwright example structure

```

from playwright.async_api import async_playwright
import asyncio

class ColabRunner:
    def __init__(self, notebook_path: str):
        self.notebook_path = notebook_path

    async def run(self):
        async with async_playwright() as p:
            browser = await p.chromium.launch_persistent_context(
                user_data_dir="./playwright-profile",
                headless=False
            )
            page = await browser.new_page()
            await page.goto("https://colab.research.google.com/", wait_until="domcontentloaded")

            await self.open_upload_dialog(page)
            await self.upload_notebook(page, self.notebook_path)
            await self.wait_until_notebook_ready(page)
            await self.run_all_cells(page)
            await self.monitor_execution(page)

            await browser.close()

    async def open_upload_dialog(self, page):
        await page.get_by_text("Upload").click()

    async def upload_notebook(self, page, file_path: str):
        file_choser_future = page.wait_for_event("filechooser")
        await page.get_by_text("Browse").click()
        file_choser = await file_choser_future
        await file_choser.set_files(file_path)

    async def wait_until_notebook_ready(self, page):
        await page.wait_for_timeout(8000)

    async def run_all_cells(self, page):
        await page.keyboard.press("Meta+F9")

    async def monitor_execution(self, page):
        for _ in range(300):
            content = await page.content()
            if "Epoch" in content or "Training" in content:
                print("Training appears active")
            await asyncio.sleep(5)

async def main():
    runner = ColabRunner("./patched_notebook.ipynb")
    await runner.run()

if __name__ == "__main__":
    asyncio.run(main())

```

5. Better production version concept

The real worker should not depend on one file and one method only.

It should use a job payload like:

```

{
  "job_id": "job_001",
  "patched_notebook_path": "/tmp/job_001.ipynb",
  "artifact_paths": [
    "model.pth",
    "metrics.json"
  ],
  "requested_gpu": true,
  "timeout_seconds": 14400
}

```

6. Session manager design

colab_session.py

Responsibilities:

- load persistent browser profile
- ensure user is already logged in
- isolate one session per active job
- handle browser crash recovery

Example:

```
from playwright.async_api import async_playwright

class ColabSessionManager:
    async def create(self):
        self.playwright = await async_playwright().start()
        self.browser = await self.playwright.chromium.launch_persistent_context(
            user_data_dir="./profiles/colab-user-1",
            headless=False
        )
        self.page = await self.browser.new_page()
        return self.page

    async def close(self):
        await self.browser.close()
        await self.playwright.stop()
```

7. Runtime manager design

Responsibilities

- detect whether runtime is connected
- request GPU runtime when needed
- wait for kernel initialization
- detect disconnected sessions

Pseudo-logic:

```
class RuntimeManager:
    async def ensure_connected(self, page):
        await page.wait_for_timeout(5000)

    async def request_gpu_if_needed(self, page):
        # Open runtime settings if your UI flow supports it
        pass
```

8. Output monitor design

The output monitor should do two things:

- capture cell text output

- detect terminal states:
 - success
 - user-code error
 - runtime disconnect
 - stuck execution
 - timeout

Example:

```
class OutputMonitor:
    async def poll(self, page):
        text = await page.locator("body").inner_text()
        return text

    def classify(self, text: str) -> str:
        if "Traceback" in text:
            return "error"
        if "Runtime disconnected" in text:
            return "runtime_disconnected"
        if "Training complete" in text:
            return "success"
        return "running"
```

9. Artifact bridge

This should prefer a deterministic artifact export strategy.

Best approach for your system:

- patch notebook so final cell copies artifacts to mounted Drive or a known export folder
- worker waits until exported artifact is visible in that location
- artifact collector downloads it from there

Example injected final cell:

```
import os
import shutil

EXPORT_ROOT = "/content/platform_export"
os.makedirs(EXPORT_ROOT, exist_ok=True)

for path in ["model.pth", "metrics.json"]:
    if os.path.exists(path):
        shutil.copy(path, os.path.join(EXPORT_ROOT, os.path.basename(path)))
```

10. Robust execution runner example

```
import asyncio
import time

class ColabExecutionRunner:
    def __init__(self, page, job_payload):
        self.page = page
        self.job_payload = job_payload
        self.started_at = time.time()

    async def execute(self):
        await self.upload_notebook()
        await self.wait_until_ready()
        await self.run_all()
        await self.watch_until_done()

    async def upload_notebook(self):
        file_chooser_future = self.page.wait_for_event("filechooser")
        await self.page.get_by_text("Browse").click()
        chooser = await file_chooser_future
        await chooser.set_files(self.job_payload["patched_notebook_path"])

    async def wait_until_ready(self):
        await self.page.wait_for_timeout(8000)

    async def run_all(self):
        await self.page.keyboard.press("Meta+F9")

    async def watch_until_done(self):
        timeout = self.job_payload["timeout_seconds"]
        while True:
            if time.time() - self.started_at > timeout:
                raise TimeoutError("Job timed out")

            text = await self.page.locator("body").inner_text()

            if "Traceback" in text:
                raise RuntimeError("Notebook execution failed")

            if "Training complete" in text:
                return

            await asyncio.sleep(5)
```

11. Important engineering rules for this worker

- never run this in your web server
- use one worker container per active job or carefully isolated sessions
- do not store Google credentials in plain text
- use a persistent profile or secure cookie/session vault
- log all automation stages
- expect selectors to break when UI changes
- add screenshot-on-failure
- add HTML dump-on-failure

12. Worker result payload

When execution ends, return structured status:


```
{
  "job_id": "job_001",
  "status": "completed",
  "stage": "execution_finished",
  "artifacts_detected": [
    "model.pth",
    "metrics.json"
  ],
  "error_type": null,
  "error_message": null
}
```

Failure example:

```
{
  "job_id": "job_001",
  "status": "failed",
  "stage": "running",
  "artifacts_detected": [],
  "error_type": "user_code_error",
  "error_message": "Traceback detected in notebook output"
}
```

Option C — Job execution engine code architecture

Now we connect backend and automation through a queue-based worker system.

1. Main goal

The job engine should:

- accept jobs
- queue them
- assign workers
- track state changes
- retry safe failures
- collect logs and results
- notify the API/frontend

2. Core engine components

```

job_engine/
├── producer/
│   └── enqueue_job.py
├── consumers/
│   ├── prepare_job_consumer.py
│   ├── execute_job_consumer.py
│   ├── artifact_consumer.py
│   └── notify_consumer.py
├── domain/
│   ├── state_machine.py
│   ├── job_service.py
│   └── retry_policy.py
├── integrations/
│   ├── notebook_service_client.py
│   ├── execution_worker_client.py
│   └── model_registry_client.py
└── events/
    ├── publishers.py
    └── handlers.py

```

3. Queue topics

Use RabbitMQ queues like:

- `job.prepare`
- `job.execute`
- `job.collect_artifacts`
- `job.register_model`
- `job.notify`
- `job.dead_letter`

4. Job lifecycle state machine

```

ALLOWED_TRANSITIONS = {
    "created": ["queued", "failed"],
    "queued": ["preparing", "cancelled"],
    "preparing": ["patched", "failed"],
    "patched": ["launching", "failed"],
    "launching": ["running", "failed"],
    "running": ["collecting_artifacts", "failed", "timed_out", "cancelled"],
    "collecting_artifacts": ["completed", "failed"],
    "completed": [],
    "failed": [],
    "timed_out": [],
    "cancelled": []
}

```

5. Producer example

When API receives `POST /jobs`, it should not execute anything directly.

It should:

1. save job in DB
2. publish `job.prepare`

```
def create_job(payload):
    job = job_repo.create(payload)
    event_bus.publish("job.prepare", {"job_id": job.id})
    return job
```

6. Prepare consumer

This consumer:

- fetches notebook version
- runs notebook analyzer if missing
- patches notebook
- updates job status
- publishes `job.execute`

```
class PrepareJobConsumer:
    def handle(self, event):
        job_id = event["job_id"]
        job_service.set_status(job_id, "preparing", "Preparing notebook")

        notebook = notebook_repo.get_for_job(job_id)
        analysis = notebook_analyzer.analyze(notebook)
        patched = notebook_patcher.patch(notebook, analysis, job_id)

        job_repo.attach_patched_notebook(job_id, patched.path)
        job_service.set_status(job_id, "patched", "Notebook patched")
        event_bus.publish("job.execute", {"job_id": job_id})
```

7. Execute consumer

This consumer:

- reserves execution slot
- sends job payload to automation worker
- updates running status
- receives result
- publishes next event

```
class ExecuteJobConsumer:
    def handle(self, event):
        job_id = event["job_id"]
        job = job_repo.get(job_id)

        job_service.set_status(job_id, "launching", "Launching execution worker")

        result = execution_client.run({
            "job_id": job_id,
            "patched_notebook_path": job.patched_path,
            "artifact_paths": job.artifact_paths,
            "requested_gpu": job.requested_gpu,
            "timeout_seconds": job.timeout_seconds
        })

        if result["status"] == "completed":
            job_service.set_status(job_id, "collecting_artifacts", "Execution finished")
            event_bus.publish("job.collect_artifacts", {"job_id": job_id})
        else:
            job_service.fail(
                job_id,
                error_type=result["error_type"],
                error_message=result["error_message"]
            )
```

8. Artifact consumer

This consumer:

- gathers output files
- stores them
- registers model
- marks job completed

```
class ArtifactConsumer:
    def handle(self, event):
        job_id = event["job_id"]
        job = job_repo.get(job_id)

        artifacts = artifact_service.collect(job_id)
        model_version = model_registry.register_from_artifacts(job_id, artifacts)

        job_repo.link_model_version(job_id, model_version.id)
        job_service.set_status(job_id, "completed", "Artifacts collected and model registered")
        event_bus.publish("job.notify", {"job_id": job_id, "type": "job_completed"})
```

9. Retry policy

Only retry platform-related failures automatically.

Retry:

- temporary browser launch failures
- temporary artifact download failure
- transient connection issues

Do not auto-retry:

- invalid notebook

- missing dependency inside user code
- notebook traceback caused by model code

Example:

```
class RetryPolicy:
    RETRYABLE = {
        "browser_start_failure",
        "runtime_disconnect",
        "artifact_fetch_temporary_failure"
    }

    def should_retry(self, error_type: str, retry_count: int) -> bool:
        return error_type in self.RETRYABLE and retry_count < 2
```

10. Job service abstraction

```
class JobService:
    def set_status(self, job_id: str, status: str, message: str):
        job_repo.update_status(job_id, status)
        job_history_repo.insert(job_id, status, message)
        redis_client.set(f"job:{job_id}:status", status)
        websocket_hub.broadcast(job_id, {
            "type": "status",
            "status": status,
            "message": message
        })

    def fail(self, job_id: str, error_type: str, error_message: str):
        job_repo.update_status(job_id, "failed")
        job_failures_repo.insert(job_id, error_type, error_message)
        self.set_status(job_id, "failed", error_message)
```

11. Concurrency control

You need a scheduler rule:

- free plan: one active job
- pro plan: multiple active jobs
- workers must reserve a slot before running

Use Redis lock:

```
def acquire_job_slot(user_id: str, max_active: int) -> bool:
    key = f"user:{user_id}:active_jobs"
    current = redis.incr(key)
    if current > max_active:
        redis.decr(key)
        return False
    return True
```

Release slot when job ends.

12. Dead letter handling

Jobs that fail unexpectedly inside consumers go to `job.dead_letter`.

Store:

- job_id
- queue name
- payload
- stack trace
- failed_at

This is very important for debugging.

13. Execution engine deployment

Recommended containers:

- api-service
- prepare-worker
- execute-worker
- artifact-worker
- notify-worker
- postgres
- redis
- rabbitmq
- minio

This lets you scale workers separately.

Option D — Frontend UI system

Now the product becomes user-facing.

Use React with a dashboard-style architecture.

1. Core pages

- login/register
- workspace dashboard
- projects list
- project detail
- notebook upload page
- notebook detail
- create job modal/page
- job detail with live logs

- model registry page
- model detail page
- usage/billing page
- settings

2. React app structure

```
frontend/
├── src/
│   ├── app/
│   │   ├── router.tsx
│   │   └── providers.tsx
│   ├── pages/
│   │   ├── auth/
│   │   ├── dashboard/
│   │   ├── projects/
│   │   ├── notebooks/
│   │   ├── jobs/
│   │   ├── models/
│   │   ├── usage/
│   │   └── settings/
│   ├── components/
│   │   ├── layout/
│   │   ├── forms/
│   │   ├── jobs/
│   │   ├── models/
│   │   ├── notebooks/
│   │   ├── charts/
│   │   └── common/
│   ├── features/
│   │   ├── auth/
│   │   ├── projects/
│   │   ├── notebooks/
│   │   ├── jobs/
│   │   ├── models/
│   │   └── usage/
│   ├── lib/
│   │   ├── api.ts
│   │   ├── websocket.ts
│   │   └── utils.ts
│   └── store/
```

3. Page-by-page UI design

Login page

Components:

- email input
- password input
- submit button
- switch to register

Dashboard page

Shows top summary:

- active jobs
- recent jobs
- recent models
- quota usage
- quick upload button

Components:

- **StatsCard**
 - **RecentJobsTable**
 - **RecentModelsList**
 - **UsageBar**
-

Projects page

Components:

- project search bar
- project grid/list
- create project modal

Each project card:

- project name
 - description
 - notebooks count
 - jobs count
 - latest run status
-

Project detail page

Tabs:

- overview
- notebooks
- jobs
- models
- settings

Overview cards:

- latest notebook
 - latest model
 - success rate
 - storage used
-

Notebook upload page

Components:

- drag-and-drop uploader
- project selector
- notebook validation preview
- submit

After upload:

- show parsed metadata
 - detected framework
 - detected artifacts
 - patch suggestions
-

Notebook detail page

Sections:

- notebook versions
- analysis output
- patch preview
- create training job button

Components:

- `NotebookVersionList`
 - `NotebookAnalysisPanel`
 - `PatchConfigForm`
-

Create job modal/page

Fields:

- execution backend
- GPU on/off

- timeout
- custom parameters
- artifact paths
- start button

Components:

- **BackendSelector**
 - **GpuToggle**
 - **JobParamsEditor**
 - **ArtifactPathEditor**
-

Job detail page

This is the most important page.

Layout:

- top job summary bar
- left: status timeline + metrics
- right: live logs + artifacts

Components:

- **JobStatusBadge**
 - **JobTimeline**
 - **LiveLogViewer**
 - **MetricsChart**
 - **ArtifactList**
 - **RetryJobButton**
 - **CancelJobButton**
-

Model registry page

Shows all models in workspace/project.

Components:

- search
- filter by framework
- model table
- compare versions button

Columns:

- model name
- framework
- latest version
- best accuracy
- updated at

Model detail page

Sections:

- metadata
- versions
- source job
- source notebook
- download buttons

Components:

- `ModelHeader`
- `VersionList`
- `LineageGraph`
- `DownloadArtifactButton`

Usage/Billing page

Components:

- jobs used today
- storage used
- active jobs
- plan card
- upgrade button

4. Important frontend state domains

Use React Query or TanStack Query for server state.

Domains:

- auth state

- workspace selection
- projects data
- notebook data
- job data
- live log stream
- model registry data
- usage data

5. Example router structure

```
<Route path="/" element={<AppLayout />}>
  <Route index element={<DashboardPage />} />
  <Route path="projects" element={<ProjectsPage />} />
  <Route path="projects/:projectId" element={<ProjectDetailPage />} />
  <Route path="notebooks/:notebookId" element={<NotebookDetailPage />} />
  <Route path="jobs/:jobId" element={<JobDetailPage />} />
  <Route path="models" element={<ModelsPage />} />
  <Route path="models/:modelId" element={<ModelDetailPage />} />
  <Route path="usage" element={<UsagePage />} />
  <Route path="settings" element={<SettingsPage />} />
</Route>
```

6. Key reusable components

- AppSidebar
- TopNavbar
- PageHeader
- StatsCard
- DataTable
- StatusBadge
- UploadDropzone
- CodeBlockViewer
- JsonViewer
- LiveLogViewer
- MetricsLineChart
- EmptyState
- ConfirmDialog

7. Live logs UI behavior

Job detail page should connect to:

- WebSocket or SSE endpoint

- auto-scroll logs
- allow pause scrolling
- filter by log level
- download logs

Component idea:

```
<LiveLogViewer
  jobId={jobId}
  autoScroll={true}
  showTimestamps={true}
  filterLevel="INFO"
/>
```

8. Metrics chart UI behavior

Use line charts for:

- train loss
- validation loss
- accuracy

Component:

```
<MetricsLineChart
  title="Training Loss"
  metricName="train_loss"
  points={points}
/>
```

9. Frontend API client structure

```
export const jobsApi = {
  create: (payload: JobCreateRequest) => api.post("/jobs", payload),
  getById: (jobId: string) => api.get(`/jobs/${jobId}`),
  getLogs: (jobId: string) => api.get(`/jobs/${jobId}/logs`),
  getMetrics: (jobId: string) => api.get(`/jobs/${jobId}/metrics`),
  cancel: (jobId: string) => api.post(`/jobs/${jobId}/cancel`)
};
```

10. Best UI flow for user

Flow A

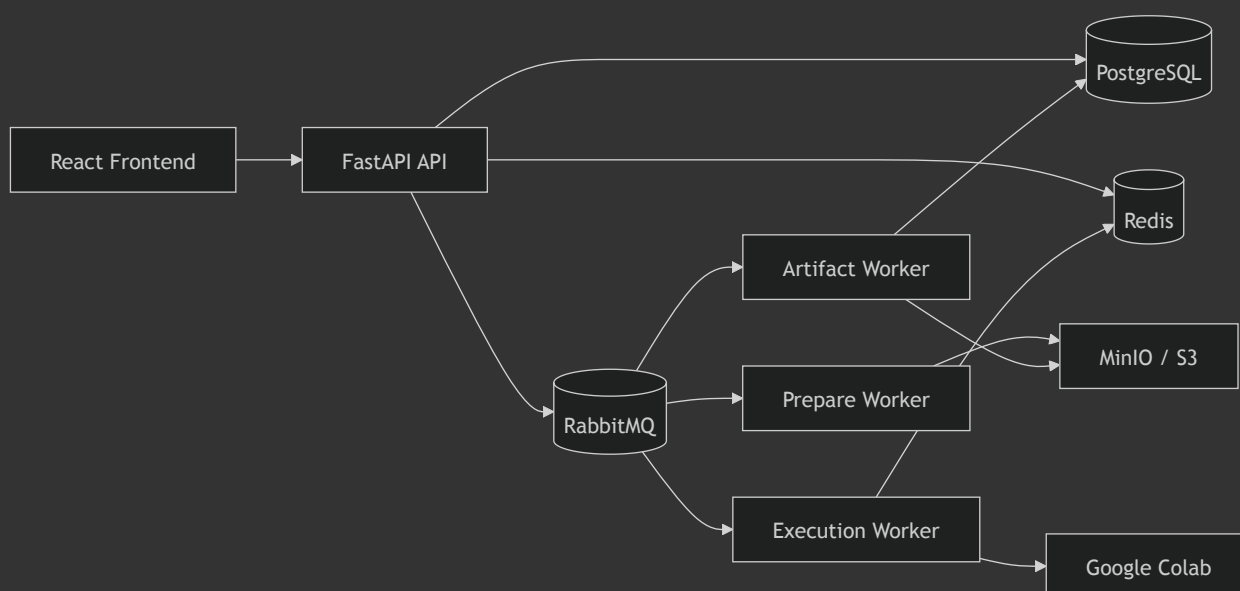
- create workspace
- create project
- upload notebook
- inspect analysis
- click train
- watch live logs

- download model

Flow B

- open existing project
- retry failed job
- compare model versions
- download best model

Final combined architecture flow



Best next step

The strongest next move is to turn this into actual starter code:

- FastAPI route skeleton
- Pydantic schemas
- RabbitMQ worker skeleton
- React page/component scaffold

I can generate that as a real project structure next.